

Toward Efficient Verification of Distributed In-Network Computing Programs

Paper # 1047, 12 pages body, 3 pages reference, 2 pages appendix

Abstract

Distributed in-network programs are increasingly deployed in data centers for their performance benefits, but shifting application logic to switches also enlarges the failure domain. Ensuring their correctness before deployment is thus critical for reliability. While prior verification frameworks can efficiently detect bugs for programs running on a single switch, they overlook the common interactive behaviors in distributed settings, thereby missing related bugs that can cause state inconsistencies and system failures.

This paper presents Procurator, a verification framework that efficiently captures interactive behaviors in distributed in-network programs. Procurator introduces a formal model combining the actor paradigm with Communicating Sequential Processes (CSP), translating pipeline execution into reactive, event-driven actors and unifying their interactions as message passing. To support flexible specification of distributed properties, it provides a unified intent language. Additionally, it incorporates a semantic-aware state pruner to reduce verification complexity, thus ensuring system scalability. Evaluation results show that Procurator efficiently uncovers 10 distinct bugs caused by interactive behaviors across five real-world in-network systems. It also reduces verification time by up to $913.2\times$ and memory consumption by up to $1.9\times$ compared to the state-of-the-art verifier.

1 Introduction

Modern data centers increasingly deploy in-network programs on programmable switches to accelerate distributed systems. These programs are typically written in P4 [6] for its flexibility and efficiency in packet processing. They often maintain application state locally while interacting with backend servers or coordinating with peer switches when deployed in a distributed manner [10, 16, 18, 19, 22, 25, 30, 38, 44, 47, 50]. As in-network programs grow in feature coverage and cross-device coordination, their complexity and hence the potential risk surface also increase [26, 32, 37], such as misforwarding,

unintended loops, or compiler/firmware-induced misbehaviors [2, 32, 37].

In practice, ensuring correct behaviors of distributed in-network programs is critical, but relying on human expertise to check the programs is both unscalable and error-prone. To address this, numerous automated verification tools have been developed to detect potential bugs and faults before deployment [11, 12, 28, 33, 40, 51, 52]. Existing work often treats distributed in-network programs as individual P4 instances. Some model the program execution as an instruction pipeline, but cannot capture the state update behaviors in typical distributed in-network programs [11, 12, 28, 33, 40]. More recent approaches extend this to a while-loop over the pipeline, enabling state tracking during verification [51, 52]. While effective for detecting single-program bugs, this abstraction fails to capture switch-server and switch-switch interactions in real deployments — the behaviors between any two programs cannot be described by a single while-loop-based modeling. As a result, the subtle bugs from the interactive behaviors are left unnoticed, leading to failure risks like state inconsistency and misforwarding.

The fundamental question is, how can we build an efficient verification framework that can detect subtle bugs arising from interactive behaviors in real-world distributed in-network programs?

To address this question, we make two key observations on distributed in-network program execution. First, a switch program behaves as a long-running process, triggered by incoming packets, performing local stateful actions, and generating output packets to interact with other switches or servers. These behaviors logically align with the actor paradigm in distributed systems [3]. Second, interactions between switch programs correspond to inter-process communication, whose properties, such as ordering and loss, can be naturally captured by the classical Communication Sequential Process (CSP) model [14]. Therefore, unlike the while-loop-based models of prior work, we propose to model distributed in-network programs as actors and their interactions as CSP.

However, realizing this idea presents several challenges.

First, distributed in-network programs feature complex P4 semantics (*e.g.*, recirculation, mirroring, state registers) that standard actor or CSP models cannot capture. To address this, we introduce a formal semantic translation that models each pipeline as a reactive event-driven actor and unifies actor interactions through message passing. This formalization produces a model suitable for verification using classical model checkers (*e.g.*, SPIN) [15]. Second, complex actor interactions produce a vast number of state updates, making it difficult to bound verification overhead to ensure scalability. We tackle this with a Semantic-Aware State Pruner that applies a tailored backward slicing algorithm. This algorithm correctly models architectural loops while aggressively pruning irrelevant states and execution paths, thereby ensuring verification efficiency. Third, existing verification intent specifications lack the flexibility to express both distributed properties (*e.g.*, consistency) and interaction topologies. We overcome this by providing a unified intent language that lets users define the network topology, state local assertions within devices, and express global invariants across devices in Linear Temporal Logic.

Specifically, we build a new verification framework called Procurator following this design. It takes as input a set of distributed in-network programs along with a verification specification written in Procurator’s Unified Intent Language. First, it constructs a program dependency graph and applies the Semantic-Aware State Pruner to optimize state spaces, thus generating an optimized intermediate representation. This representation is then processed by the Semantic Translation Model, which converts it into a unified actor-and-CSP-based model that can be verified by a classical model checker (*e.g.*, SPIN). Finally, the distributed in-network program verification is completed.

We implemented a prototype of Procurator and will release it upon the publication of this manuscript. It consists of $\sim 12k$ lines of C++ and $\sim 2k$ lines of Python, and supports the V1Model and TNA in-network programming frameworks for software and Tofino switches. The prototype integrates with the standard P4 toolchain via the p4c front-end compiler and leverages the SPIN model checker for backend verification. We conduct extensive experiments on five real-world distributed in-network systems with interactive behaviors, as well as four single-device in-network programs. The results show that Procurator successfully uncovers 10 distinct bugs arising from interactive behaviors. For single-device programs, it matches the bug-detection capability of the state-of-the-art tool p4tv [52] while reducing verification time by up to $913.2\times$ and memory consumption by up to $1.9\times$.

2 Background and Motivation

We start by presenting the background of verifying distributed in-network programs and analyzing the typical behaviors of these systems. We then discuss our design choice.

2.1 Background

Distributed in-network programs. With the nature of in-network and line-rate processing, programmable switches are increasingly used to offload parts of the logic of distributed systems [1, 6, 10, 13, 16–19, 22, 24, 25, 29, 30, 38, 44–47, 49, 53]. For instance, Gecko [17] implements a P4-based load balancer on ToR switches to redirect read and write requests to distributed key-value storage servers. In addition, in-network P4 programs can be deployed across multiple switches; for example, NetChain offloads key-value store caches to the on-chip memory of a chain of switches [18]. Such systems always rely on streaming-based interactions, either between switches and backend servers or directly among switches across the network.

P4 program verification. In-network P4 programs must be verified before deployment to prevent system failures caused by unexpected program errors. For stateless programs, existing verification methods typically model program execution as a pipeline of instructions, with inputs and outputs represented as mathematical symbols [11, 12, 28, 31, 33, 40, 42]. The output is then verified against predefined assertions to ensure correctness. For stateful programs, recent works like P4Inv [51] and p4tv [52] introduce a while-loop in the instruction pipeline, iteratively updating states in hardware registers, but conduct verification based on mathematical symbols and state-machine-based model checking [9].

While these approaches are effective in verifying individual P4 programs, they overlook the potential errors arising from the interactive behaviors in distributed in-network programs. In the following, we showcase such issues.

2.2 Motivating Examples

In this section, we use two typical examples to illustrate the inherent error-prone behaviors in distributed in-network programs, caused by interaction between switches and backend servers or directly among switches.

2.2.1 Switch-Server Interaction

In many in-network systems [7, 17, 30, 38, 50], switches must redirect client requests to multiple backend servers (*a.k.a.* multicast fan-out). This is typically implemented by writing P4 programs that send the first copy of a packet on its initial pass and use *recirculation* to emit additional copies on subsequent passes. However, this mechanism can introduce inconsistent request ordering across backend servers. To illustrate this, we consider a switch T and two backend key-value replicas, S_1 and S_2 : T delivers each write request to one server directly (*a.k.a.* the first pass $p1$), where the server is chosen randomly based on load balancing (LB), and then uses *recirculation* to send a copy to the other (*a.k.a.* the second pass $p2$).

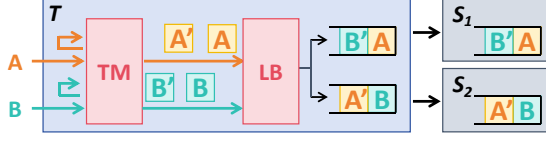


Figure 1: An example of *Replica Total Order* violation.

```

1  idx = get_index(hdr.key);
2  local_seq = sequence_reg[idx];
3  if (head_node) {
4    local_seq = local_seq + 1;
5    hdr.seq = local_seq;
6  }
7  if (head_node || hdr.seq > local_seq) {
8    sequence_reg[idx] = hdr.seq;
9    value_reg[idx] = hdr.value;
10 } else {
11   drop();
12 }
13 routing();

```

Figure 2: Code snippet for the key functionality of NetChain [18].

As shown in Figure 1, two write requests, *A* and *B*, arrive at *T* back-to-back. When packet *A* arrives at the traffic manager (TM), the traffic manager flags it for recirculation. However, the recirculation operation is not atomic. Therefore, TM then starts to handle packet *B*, and flags it for recirculation. Thus, LB receives data packets in the order of *A*, *B*, *A'*, and *B'*. For *A* and *B*, the load balancer sends them to *S*₁ and *S*₂, respectively, while *A'* and *B'* are sent to *S*₂ and *S*₁, respectively, according to functional requirements.

Clearly, *S*₁ observes the order *A* → *B*, while *S*₂ observes *B* → *A*, which violates consistency between replicas. The root cause of this issue is architectural: recirculation splits a single logical broadcast into multiple sequential sends, which can easily lead to error-prone interleaving when multiple broadcasts coexist. While developers' expertise could help prevent such issues, this behavior still requires verification before deployment to production environments to ensure reliability.

2.2.2 Switch-Switch Interaction

We use NetChain [18] as a case study to illustrate potential errors arising from switch-switch interactions. NetChain stores cache replicas across a chain of switches in a key-value system, maintaining consistency via sequence numbers. As shown in Figure 2, when the head switch receives a write request, it increments its local sequence number, stores the data, and forwards both the data and sequence number to the subsequent switches. Each subsequent switch updates its local replica only if the incoming sequence number is larger than its local one, ensuring consistency across the chain.

NetChain's design has been fully verified using TLA+ [21], yet errors can still arise from switch-to-switch interactions, of-

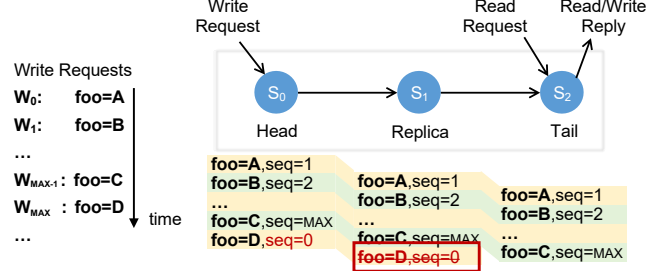


Figure 3: An example of a distributed P4 system bug.

ten overlooked by prior verification methods. Figure 3 shows an example of such an error.

Switches *S*₁, *S*₂, and *S*₃ form a chain, with *S*₀ as the head and *S*₂ as the tail. The system begins with no data stored, and *W*_{*i*} denotes a write request, while *seq* (ranging from 0 to MAX) tracks sequence numbers. When *W*₀ arrives, all switches set *foo* to *A* and *seq* to 1. Upon receiving *W*₁, *S*₀ updates *foo* to *B*, increments *seq* to 2, and forwards the request along with the updated *seq*. Both *S*₁ and *S*₂ update *foo* to *B* and set *seq* to 2, preserving consistency.

However, when *seq* reaches MAX, the behavior starts changing. Upon receiving *W*_{MAX-1}, all switches update *foo* to *C* and set *seq* to MAX. When *W*_{MAX} arrives, *S*₀'s *seq* overflows, wrapping around to zero (lines 3-4 in Figure 2). It forwards *W*_{MAX} = *D* with *seq* = 0, but *S*₁ and *S*₂, having *seq* = MAX, ignore the update. Therefore, the data in *S*₁ conflicts with *S*₀, and this consistency bug cannot be detected by verifying each program individually due to cross-switch interactions and version wrap-around.

2.3 Our Design Choice

In §2.2, we demonstrate errors caused by switch-server, switch-switch interactions, and P4-specific features (e.g., recirculation) in distributed in-network programs. This makes it urgent to design a verification approach that efficiently captures these behaviors.

A straightforward solution might be to extend existing stateful P4 verification tools (e.g., P4Inv [51], p4tv [52]) to account for interactive behaviors in distributed in-network programs. However, this approach is fundamentally infeasible. Existing tools treat P4 features like recirculation and mirroring as iterations within a while loop, making them incapable of detecting logical errors that occur between loops — the root cause of the issues in §2.2. Moreover, the while-loop modeling lacks visibility into external interactions (with other switches or servers), making direct extension of these tools impractical.

Therefore, we must design a new and systematic verification approach. Our core insight is that a distributed in-network program consists of independent, reactive computational units communicating over network channels. Based on this, we leverage two classical techniques that align well with these behaviors:

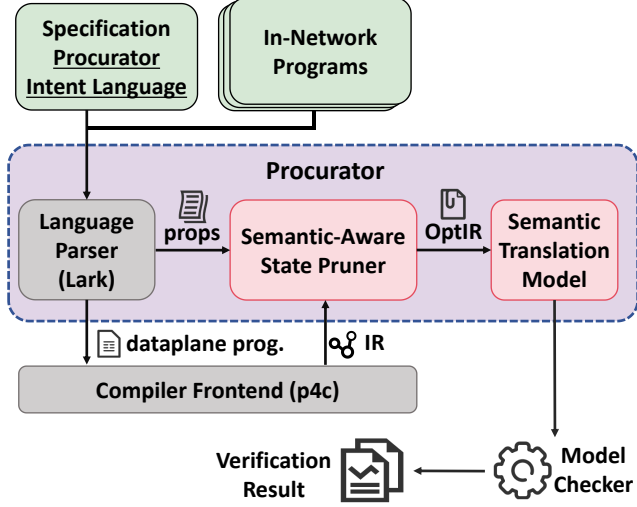


Figure 4: The workflow of Procurator. Procurator accepts the specification and in-network programs as input. *props* denotes the verification intent-related properties, and *dataplane prog.* are dataplane programs. *IR* and *OptIR* are the original and optimized intermediate representations, respectively.

The Actor paradigm. A switch operates as a continuously running process, reacting to packet arrivals, performing stateful actions, and generating events (*e.g.*, sending packets). This aligns with the Actor paradigm [3], where an actor encapsulates its state and interacts via message passing. An actor performs operations based on the incoming message: (1) triggers a process handler to manipulate its own states; (2) interacts with others by sending messages. This approach has proven efficient in complex distributed applications [20, 34], making it ideal for modeling distributed in-network programs, in contrast to the traditional sequential while-loop approach.

The Communicating Sequential Process (CSP) model. Each switch program is an independent process, and the packet exchanges between them constitute inter-process communication. The CSP [14] model is a classic framework that describes and analyzes such concurrent process interactions, allowing us to explicitly define semantics like ordering and loss, which are crucial for distributed in-network programs.

Thus, our approach, Procurator, chooses to model the pipeline distributed in-network programs using the actor paradigm, with interactions captured through the CSP model.

3 Procurator Workflow

As depicted in Figure 4, Procurator takes two inputs: a set of in-network programs, as well as a user-written specification based on Procurator Intent Language. Its workflow is built on three core components that process these inputs to produce a verifiable formal model.

Language Parser. This component serves as the entry point for the user’s Specification. It is a lightweight parser based on the Lark [39] toolkit. It parses the file to extract the core Verification Intent, which specifies the properties to be checked. This intent provides the precise goal for the subsequent optimization step. Meanwhile, it also calls the compilation front end (in our implementation, p4c) to process the dataplane information, providing a basis for subsequent pruning and optimization. We describe the syntax and semantics of the intent language in detail in §6.

Semantic-Aware State Pruner. Concurrently, the in-network programs are processed by the compiler frontend (p4c) to generate an Intermediate Representation (IR). The State Pruner then acts as Procurator’s optimization engine. As shown in Figure 4, it receives two inputs: the verification intent-related properties from the language parser and the IR from the compiler. Based on this intent, it identifies and eliminates irrelevant program logic, generating a smaller, optimized IR. We detail this pruning algorithm in §5.

Semantic Translation Model. Ultimately, Procurator transforms the optimized IR into a Semantic Translation Model. The Semantic Translation Model is a two-layer model proposed by Procurator. It simulates the interaction behaviors between different devices through CSP and the complex semantic behaviors within the devices through actors. We described this model in detail in §4.

4 Semantic Translation Model

Procurator introduces a semantic translation model that maps P4 semantics in distributed in-network programs to actor and CSP representations. It formalizes two key aspects to capture the interactive behaviors: (1) the reactive, event-driven nature of the pipeline, modeled as a reactive actor; and (2) diverse packet interaction primitives, unified under a message-passing abstraction. Appendix §A has more details on the abstraction of parsing and deparsing rules to support the conditional and mirror semantics.

4.1 Modeling the Pipeline as a Reactive Actor

Procurator models the pipeline as a *Reactive Actor* driven by two internal channels: an ingress channel *I* (IngressChan) and an egress channel *E* (EgressChan). This design captures the event-driven nature of switches responding to packet arrivals and naturally models self-messaging primitives such as *recirculate*. The actor follows an *Ingress-Triggered, Egress-Draining* model, where each packet arrival initiates a complete reaction: the ingress packet arrival triggers pipeline processing, and any resulting operations are drained from the egress channel before the next reaction begins.

Preliminaries. The *actor state* is defined as a pair (S, I) , where *S* represents the program-visible dataplane state, including packet headers, metadata, and persistent structures like registers, and *I* is the FIFO ingress channel

(IngressChan). The egress channel E is treated as a transient argument to the draining judgment, not as part of the persistent actor state.

The pipeline logic is abstracted by two atomic functions:

- $\Pi_I(S, p) = (S', E_{\text{out}})$: atomic ingress execution on input packet p , returning an updated state S' and an initial egress channel E_{out} .
- $\Pi_E(S, I, p) = (S', I', L_p, E_{\text{add}})$: atomic egress execution on packet p . It may update the dataplane state to S' , the ingress channel to I' (for recirculation), and returns both a list of external events L_p and a list of packets E_{add} to be prepended to the egress channel (for i2e and e2e cloning).

Channel (FIFO Queue) Operations. All channels in the model are FIFO queues. We define the core operations: (1) $[]$: an empty channel; (2) $\text{enq}(Q, p)$: Add element p to queue Q ; (3) $\text{deq}(Q)$: dequeuing the head element p of a non-empty queue Q ; (4) $\text{is_empty}(Q)$: checking if Q is empty.

Observable External Events. An observable external event is defined as enqueueing a packet into any ingress channel I . This captures two behaviors: (1) standard forwarding, where the packet from an actor is enqueued to another actor's ingress channel, (2) *recirculate*, where the packet is enqueued back to the same actor's ingress channel. In contrast, an *enq* operation on an egress channel E is part of the actor's internal processing and does not constitute an observable event.

We denote an external event as a pair (c, p') , representing the operation $\text{enq}(c, p')$, where c is the target ingress channel. The symbol L represents a finite sequence of such events.

Operation Semantics. The core of our model is the complete, atomic reaction of an actor to a single packet, defined by Rule (1). This process is triggered when the actor's ingress channel I is not empty. It uses the *deq* operation to atomically retrieve the head packet p_{in} and the remaining ingress channel I_0 . The packet p_{in} is then processed by the ingress pipeline, and the results are fully drained from the egress channel, producing a sequence of external events L . Finally, the actor's state is updated with the new ingress channel I_0 .

$$\frac{(p_{\text{in}}, I_0) = \text{deq}(I) \quad \langle S_0, p_{\text{in}} \rangle \xrightarrow{\Pi_I} \langle S_1, E_1 \rangle \quad \langle S_1, E_1 \rangle \xrightarrow{\text{Drain}} \langle S_2, L \rangle}{\langle S_0, I \rangle \xrightarrow{L} \langle S_2, I_0 \rangle} \quad (1)$$

The rule (1) relies on a key helper process, *Drain*, which recursively processes all packets in the egress channel. This process is defined by the following two rules. Rule (2) is the base case: if the channel is empty, the process terminates.

$$\frac{\text{is_empty}(E) = \text{true}}{\langle S, I, E \rangle \xrightarrow{\text{Drain}} \langle S, I, [] \rangle} \quad (2)$$

Rule (3) is the recursive step, now enhanced to support packet re-injection. It dequeues a packet p_e , processes it via Π_E , and obtains a list of packets to re-inject, E_{add} . These

packets are prepended to the remaining queue E_0 to form the new queue E' for the recursive call.

$$\frac{(p_e, E_0) = \text{deq}(E) \quad \langle S_0, I_0, p_e \rangle \xrightarrow{\Pi_E} \langle S_1, I_1, L_p, E_{\text{add}} \rangle \quad E' = \text{concat}(E_{\text{add}}, E_0) \quad \langle S_1, I_1, E' \rangle \xrightarrow{\text{Drain}} \langle S_2, I_2, L_0 \rangle}{\langle S_0, I_0, E \rangle \xrightarrow{\text{Drain}} \langle S_2, I_2, L_p \cdot L_0 \rangle} \quad (3)$$

This model effectively divides packet processing into two phases. The ingress pipeline (Π_I) acts as an event handler, consuming one packet via the top-level *Actor-React* rule (Rule 1). The egress pipeline (Π_E), invoked via the *Drain* process (Rules 2 and 3), then executes all resulting actions to produce the final, observable external events. The specific behaviors of Π_I and Π_E are determined by the semantics of the *TrafficManager* and *FinalDispatch* stages, which we formalize in the next section §4.2.

4.2 Unifying Interaction as Message Passing

We unify all interactive behaviors that move a packet across logical boundaries, such as forwarding, cloning, and recirculation, and abstract them as message passing between actors. This abstraction is captured by the semantics of two architectural stages: *TrafficManager* (TM) and *FinalDispatch* (FD). These stages interpret packet metadata to resolve the programmer's intent, deciding whether to deliver the message to an external actor (forwarding) or back to the same actor (recirculation).

4.2.1 TrafficManager (TM) Semantics

The *TrafficManager* is the final logical stage of the ingress pipeline Π_I . Its responsibility is to populate the egress channel (E) based on the packet's metadata, particularly *clone_spec*. Its behavior is defined as a function $\text{TM}(p) = E_{\text{out}}$. We also define the helper functions for cloning:

- *clone*(p): This function creates an exact copy of a packet, p_{clone} .
- *clone_and_mark_i2i*(p): This function performs the *clone*(p) operation and additionally sets a metadata flag on the clone: $p'_{\text{clone}}.\text{meta}.\text{is_i2i_clone} \leftarrow \text{true}$.

Rule (4) defines the default case. When no special cloning is required, the TM creates a new egress channel containing only the current packet.

$$\frac{p.\text{meta}.\text{clone_spec} = \text{None}}{\text{TM}(p) = \text{enq}([], p)} \quad (4)$$

The rules for other cloning types (e.g., ingress-to-egress) and packet drops within the ingress pipeline are detailed in Appendix §B.

4.2.2 FinalDispatch (FD) Semantics

The `FinalDispatch` is the final logical stage of the egress pipeline Π_E and the sole exit point for all packets. Its responsibility is to translate a packet p being processed in the egress pipeline into a tuple (L_p, E_{add}) , where L_p is a list of formal representations of external `enq` events, and E_{add} is a list of packets to be re-injected into the egress pipeline.

The behavior of `FD`, particularly for forwarding, is critically dependent on the network topology, which we abstract as a `Links` relation. This relation is a set of tuples of the form (Source Actor, Egress Port, Destination Channel). If a packet's destination does not match any of the following rules, it is implicitly dropped (i.e., $\text{FD}(p) = ([], [])$), with further details in Appendix §C.

Rule (5) defines standard, topology-aware forwarding. An external event is generated only if a valid link exists in the `Links` relation.

$$\frac{p.\text{meta}.\text{recirculate_flag} = \text{false} \quad (X, p.\text{meta}.\text{egress_port}, c_{\text{dest}}) \in \text{Links}}{\text{FD}(p) = ([c_{\text{dest}}, p'], [])} \quad (5)$$

Rule (6) models recirculation. This is an internal behavior that generates an event targeting the actor's own ingress channel, I_{self} .

$$\frac{p.\text{meta}.\text{recirculate_flag} = \text{true}}{\text{FD}(p) = ([I_{\text{self}}, p'], [])} \quad (6)$$

Rule (7) models egress-to-egress (E2E) cloning. It produces one external event for the original packet and re-injects the clone packet into the egress pipeline, allowing the clone to have its own lifecycle.

$$\frac{p.\text{meta}.\text{clone_spec} = \text{E2E} \quad p_{\text{clone}} = \text{clone}(p) \quad (X, p.\text{meta}.\text{egress_port}, c_{\text{dest}}) \in \text{Links}}{\text{FD}(p) = ([c_{\text{dest}}, p'], [p_{\text{clone}}])} \quad (7)$$

The rule models for the ingress-to-ingress mirroring, and packet drops within the egress pipeline are detailed in Appendix C.

Model Boundary Discussion. It is worth noting that since $\text{clone}(p)$ is an exact copy, our model allows a cloned packet to be cloned again, which could theoretically lead to non-termination. However, in practical network applications, protocol designs that require infinite mirroring are exceedingly rare. We therefore do not explicitly model a termination condition in order to maintain the simplicity of the core model.

4.3 Executable Translation for Promela

To automate the verification of distributed properties, we translate our formal operational semantics into an executable model in Promela, the specification language of SPIN [15]. Promela's `proctype` and `chan` directly instantiate our global actor model: devices become concurrent processes; inter-device links are FIFO channels. We now give a translation

that preserves the reaction-level semantics (§4.1) and the observable interface (*enqueue into any ingress*).

System composition in Promela. The parallel composition $\text{System} = D_1 \parallel \dots \parallel D_n$ becomes the `init` block. We declare one *observable* ingress channel per device and spawn a `proctype` per device, passing its ingress handle.

```
init {
  chan s1_ig_chan = [Q_IN] of { packet_t };
  chan s2_ig_chan = [Q_IN] of { packet_t };

  run device_proc(1, s1_ingress);
  run device_proc(2, s2_ingress);
}
```

Device pipeline as a *proctype*. Each device implements the Reactor pattern and executes a whole *reaction* inside one `atomic{}`: dequeue one ingress packet, run Π_I , populate the local egress queue via TM, then *drain* the egress queue by repeatedly running Π_E and dispatching via `FD`. The reaction ends only after the egress queue becomes empty, matching Rule (2) and (3).

```
proctype device_proc()
{
  chan egress_q = [Q_EG] of { packet_t };
  packet_t p;
  packet_t pe;
  do
  :: ingress_chan ? p ->
    atomic {
      IngressParser(p);
      Ingress(p);
      IngressDeparser(p);
      TrafficManager(p);
      do
      :: len(egress_q) > 0 ->
        egress_q ? pe;
        EgressParser(pe);
        Egress(pe);
        EgressDeparser(pe);
        FinalDispatch(self,
          my_ingress, pe);
      :: else -> break
      od;
    }
  od
}
```

Traffic Manager inline function. To realize Rule (1), we materialize Π_I 's E_{out} as pure *local* appends to the device egress queue: 'drop' elides the packet, 'I2E/I2I' duplicate locally and append, and no channel send occurs; thus TM yields no observables within one reaction.

```

inline TrafficManager(packet_t p)
{
    if
    :: p.drop_flag ->
        skip
    :: else ->
        if
        :: p.clone_spec == I2E ->
            packet_t c = p;
            c.egress_spec = get_mirror_port(p);
            egress_q ! p;
            egress_q ! c;
        :: p.clone_spec == I2I ->
            packet_t c2 = p;
            c2.is_i2i_clone = true;
            egress_q ! p;
            egress_q ! c2;
        :: else ->
            egress_q ! p
        fi
    fi
}

```

Final Dispatch inline function. Consistent with Rules (5, 6, 7), FD deterministically maps flags to a single observable action: `recirc/is_i2i_clone` (with capacity asserts) self-enqueue to `my_ingress`; otherwise it forwards via `egress_spec`. FD is the sole boundary where internal effects become channel sends. See Appendix D for `send_to()`.

```

inline FinalDispatch(packet_t pe)
{
    if
    :: pe.drop_flag ->
        skip
    :: pe.recirc_flag ->
        assert(len(my_ingress) < Q_IN);
        my_ingress ! pe
    :: pe.is_i2i_clone ->
        assert(len(my_ingress) < Q_IN);
        my_ingress ! pe
    :: else ->
        send_to(self, pe.egress_spec, pe)
    fi
}

```

5 Semantic-Aware State Pruner

A key challenge in verification is state-space explosion, where the number of possible states grows exponentially with concurrent actors and their interactions. Directly executing our actor-based model in a model checker can generate an enormous state space, resulting in high computational overhead.

To tackle this, we propose a semantic-aware state pruner, which aims to simplify the intermediate representation (IR)

Algorithm 1 Semantic-Aware DDG Construction from CFG

Require: $CFG = (N, E, entry, exit)$

Ensure: $DDG = (N, E_D)$

```

1: function BUILDDDG( $CFG$ )
2:    $G \leftarrow$  empty directed graph
3:    $RD_1 \leftarrow$  REACHINGDEFS( $CFG$ )
4:   for all  $u \in N$  do
5:     for all  $x \in \text{USES}(u)$  do
6:        $D_1[u, x] \leftarrow \{d \in N \mid \text{DEFINES}(d, x) \wedge d \in$ 
7:          $RD_1[u]\}$ 
8:       for all  $d \in D_1[u, x]$  do
9:         ADDEDGE( $G, d, u, x$ )
10:      if NEEDSLOOPEDGE( $CFG$ ) then
11:         $CFG^+ \leftarrow (N, E \cup \{(exit, entry)\}, entry, exit)$ 
12:         $RD_2 \leftarrow$  REACHINGDEFS( $CFG^+$ )
13:        for all  $u \in N$  do
14:          for all  $x \in \text{USES}(u)$  do
15:             $D_2 \leftarrow \{d \in N \mid \text{DEFINES}(d, x) \wedge d \in$ 
16:               $RD_2[u]\}$ 
17:             $New \leftarrow D_2 \setminus D_1[u, x]$ 
18:            for all  $d \in New$  do
19:              if CROSSOK( $x$ ) then
20:                ADDEDGE( $G, d, u, x$ )
21:      return  $G$ 

```

```

20: function CROSSOK( $x$ )
21:   return ISREGISTER( $x$ )  $\vee$  ISPACKETCARRIED( $x$ )

```

and reduce model-checking time. The core idea is to eliminate states that do not affect the properties being verified.

However, traditional static analysis tools, which operate on static Control Flow Graphs, treat the P4 pipeline as a single-pass execution graph. This view cannot capture primitives like recirculation, where a packet exits the pipeline only to be reinjected for a subsequent pass. The root cause of this issue is the failure to recognize the “cross-pass data dependency”: the state modified in one pass becomes a critical input that governs the control flow of the next pass.

Dependency Graph Construction. Specifically, we propose a semantic-aware data dependency graph construction algorithm (Algorithm 1) that takes only the program’s control flow graph (CFG) as input and produces a data dependency graph (DDG). The algorithm proceeds in two passes. In the first pass, it runs a classical *reaching-definitions* analysis on the original CFG (line 3) and creates a baseline DDG by adding one edge $d \rightarrow u$ labeled with variable x whenever x is used at node u and a definition of x at node d reaches u (line 8). If the program contains a pipeline loop (e.g., due to `recirculate/mirror`), the second pass materializes it by inserting a *single* back edge from `exit` to `entry` (line 10), recomputes reaching definitions on the augmented CFG (line 11), and then adds only those *new* cross-iteration dependency edges that did not exist

Algorithm 2 Backward Slicing-based Pruning

Require: Data-dependency graph $DDG = (V, E_D)$; Control-dependency graph $CDG = (V, E_C)$; seed nodes $Seeds \subseteq V$

Ensure: Set of kept nodes $Keep \subseteq V$

```
1: procedure BACKWARDPRUNE( $DDG, CDG, Seeds$ )
2:    $Keep \leftarrow \emptyset$ ;  $Work \leftarrow Seeds$ 
3:   while  $Work \neq \emptyset$  do
4:      $u \leftarrow \text{POP}(Work)$ 
5:     if  $u \in Keep$  then
6:       continue
7:      $Keep \leftarrow Keep \cup \{u\}$ 
8:     for all  $(p \rightarrow u) \in E_D$  do
9:        $\text{PUSH}(Work, p)$ 
10:    for all  $(c \rightarrow u) \in E_C$  do
11:       $\text{PUSH}(Work, c)$ 
12:  return  $Keep$ 
```

after the first pass (line 15). Each such edge is guarded by a boolean predicate $\text{CROSSOK}(x)$ (line 18) that authorizes cross-iteration carry of x (true for persistent state, such as registers, or for packet-carried fields).

Algorithm 2 performs a backward slicing over the union of dependencies: starting from the property-related seeds, it traverses all reachable statement nodes via data edges and control edges. All nodes that are backward-reachable constitute $Keep$ and are retained; others can be pruned.

In Figure 5, the code exhibits a pipeline loop (e.g., a `recirculate`) where a field updated in one pass decides a branch and triggers an action in the next pass. When the DDG already encodes this with the single $exit \rightarrow entry$ back edge from the previous section, the backward traversal naturally climbs from the seed location in the second pass to the writing site in the first pass via DDG, while CDG preserves the necessary branch conditions along the way. By contrast, Figure 8b illustrates the failure of the single-pass dependency view: missing the cross-iteration link, the traversal cannot reach the prior definition and wrongly drops key statements. Our approach (shown in 8c) recovers the cross-iteration chain and keeps exactly those statements.

Analysis. After pruning, the state in IR retains only the minimal set of packet headers, metadata fields, and registers that can influence property verification. Likewise, the CFG preserves only conditional branches that affect the execution of statements deemed essential in the data dependency graph. Execution paths irrelevant to these state variables are eliminated. As a result, both the state vector size and the overall state space are significantly reduced, alleviating the computational burden of subsequent model checking.

```
1 if (hdr.ipv4.dst_ip == 192.168.0.1) {
2   meta.egress_port = 32;
3 } else {
4   hdr.ipv4.dst_ip = 192.168.0.1;
5   recirculate();
6 }
```

Figure 5: An example of a P4 backward dependency.

```
 $spec ::= decl^+$ 
 $decl ::= \text{import name from string [ entries string ] ;}$ 
 $::= | topo$ 
 $::= | local$ 
 $::= | global$ 

 $topo ::= \text{topology link}^*$ 
 $link ::= \text{link dev} \rightarrow \text{dev (port\_id | ALL) ;}$ 

 $local ::= \text{local dev local\_stmt}^*$ 
 $local\_stmt ::= \text{let id = expr ;}$ 
 $::= | \text{assert expr ;}$ 

 $global ::= \text{global ltl\_rule}^*$ 
 $ltl\_rule ::= \text{ltl name ltl\_expr ;}$ 
 $ltl\_expr ::= \text{temporal\_op expr}$ 
 $::= | ltl\_expr \text{ bin\_op } ltl\_expr$ 
 $::= | \text{un\_op ltl\_expr}$ 
 $::= | ( ltl\_expr )$ 

 $temporal\_op ::= '[]' | '<>' | 'X' | 'U'$ 

 $expr ::= id | literal$ 
 $::= | \text{expr arith\_op expr}$ 
 $::= | \text{expr logic\_op expr}$ 
 $::= | id . field$ 
```

Figure 6: The abstract syntax of the Procurator Intent Language.

6 Procurator Intent Language

We now introduce the Procurator Intent Language for specifying verification properties of distributed in-network programs with complex interactive behaviors. It unifies three essential components: (1) topology behavior specification, defining switches and their interconnection topology; (2) local behavior specification, specifying contracts and assertions in a single switch; and (3) global behavior specification, expressing temporal properties across multiple switches.

A Procurator intent specification consists of a (topology, local intent, global intent) triple. Figure 6 presents the syntax.

Topology behavior specification. As shown in the syntax for *topo* and *link*, the `topology` block represents the network as a directed graph. Each `link` statement is closely tied to the

```

1 import s0 from "netchain_16.p4" entries "commands_0.txt";
2 import s1 from "netchain_16.p4" entries "commands_1.txt";
3 import s2 from "netchain_16.p4" entries "commands_2.txt";
4
5 topology {
6   link s0 -> s1 ALL;
7   link s1 -> s2 ALL;
8 }
9
10 global {
11   ltl a {
12     [] {
13       (s0.sequence_reg[0] >= s1.sequence_reg[0])
14       &&
15       (s1.sequence_reg[0] >= s2.sequence_reg[0])
16     };
17   };
18 }

```

Figure 7: Specification S of NetChain in Procurator.

FinalDispatch stage (§4.2.2). The declaration `link X -> Y port_id` provides routing information to the code generator: when switch X’s actor forwards a packet from logical port `port_id`, the generator translates it into a message sent to switch Y’s IngressChannel using Rule(5). The syntax also supports the `ALL` keyword, which specifies a default, unconditional forwarding path.

Local behavior specification. The `local` block enables engineers to specify functional contracts for the program in a single switch. According to the `local_stmt` rule in Figure 6, it supports two statement types:

- **Variable Declaration and Computation:** Using the `let` keyword, engineers can declare temporary variables and assign them the result of `expr`. This expression can involve arithmetic and logical operations on state variables like headers or registers.
- **Assertion:** Using the `assert` keyword, engineers can check whether a boolean `expr` holds true.

Assertions are automatically injected just before the FinalDispatch stage, allowing precise checks on the pipeline state before generating external events. Variables declared with `let` are scoped to the local block but can also be referenced in the `global` block (for global behavior specification), enabling verification logic that spans both local computations and global properties. During parsing, Procurator performs semantic checks to ensure all variables originate from the actor’s state or are locally declared.

Global behavior specification. The `global` block specifies system-wide properties using Linear Temporal Logic (LTL). The syntax for `ltl_rule` and `ltl_expr` defines named LTL properties, supporting standard temporal operators such as `[]` (always), `<>` (eventually), `X` (next), and `U` (until).

LTL expressions can reference state variables from any actor (*e.g.*, headers, metadata, or registers) as well as variables declared in `local` blocks. During parsing, Procurator

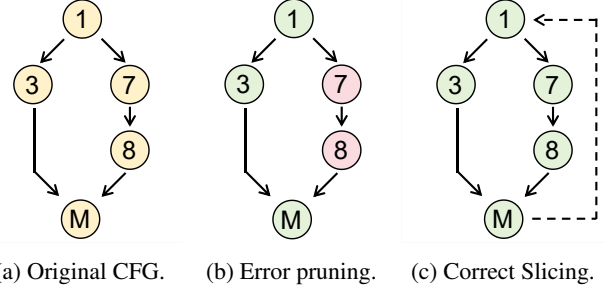


Figure 8: CFG of example in Figure 5.

performs semantic checks to ensure all variables are valid, preventing specification errors.

A concrete example. We now provide a concrete example to verify the sequence monotonicity property in NetChain. As shown in Figure 7, the `topology` block explicitly defines the linear structure `s0->s1->s2`. The `global` block specifies a single LTL property, `sequence_monotonicity`, which uses the always operator (`[]`) to assert an invariant: at all times, the `sequence_reg` value in an upstream switch must be greater than or equal to that in its downstream neighbor. This cross-actor state assertion captures a distributed property that traditional verifiers for the program on an individual switch cannot express.

7 Implementation

We implement a prototype of Procurator using ~12k lines of C++ and ~2k lines of Python. Our prototype includes a language parser for our unified intent language, a semantic translation model that generates verifiable outputs by the model checker, and a semantic-aware state pruner for optimization.

Procurator Intent Language Parser. To handle user-defined verification tasks, we implemented an interpreter for Procurator Intent Language, leveraging the Lark [39] toolkit. This component is responsible for processing a specification file that defines the system’s topology, local invariants, and global properties. The output of this parser is a structured representation of the verification intent, which guides both the state pruner and the final model generation.

Semantic Translation to Promela. The semantic translation model converts the in-network program’s intermediate representation (IR) into a verifiable model for the SPIN [15] model checker. We implemented this translator as a custom backend for the p4c [35] in C++. The translator methodically maps P4’s operational semantics to the concepts of Communicating Sequential Processes (CSP), which are natively supported by Promela [15].

Semantic-Aware State Pruner. Our implementation of the pruner functions as an optimization pass that simplifies the P4 compiler’s IR before it is fed to the Promela translator. The pruner implements the backward slicing algorithm detailed in §5, which constructs a precise data dependency graph based

System	Property	Root Cause	States	Time	LoC of Host and Spec
Netchain [18]	Sequence monotonicity	Sequence number wraparound	229374	1min39s	162
P4xos [10]	Majority quorum	Incorrect quorum computation due to persistent stale counts	10765	1min27s	141
	Register access safety	Out-of-bound register access*	4507	35.3s	108
	Forwarding correctness	Drop_flag not being correctly set*	53	0.45s	93
Gecko [17]	Timer Packet Loss or Blockage	Single timer packet is lost, halting timeout updates	8526	41.6s	515
	Limited Concurrency Handling	Timer logic can only process one request's per traversal	1348	4.68s	516
	Improper Timer Initialization	Early write requests, bypassing in-switch logic	2679	9.17s	520
DistCache [30]	Integer Wrap-around in Load Tracking	leafload variable overflows, causing misrouting	163858	18.5s	162
ATP [22]	Indefinite Aggregation	8-bit agtr_time set above intended range	72	0.17s	129
	Aggregation Count Mismatch	Packet loss leads to fewer completed	390023	5min30s	148

Table 1: Effectiveness and efficiency in detecting bugs. The "LoC of Host" column indicates the lines of Promela host-model code required for each system. Results shown here assume no additional optimizations in place. (* indicates old bugs previously found by existing P4 verifiers.)

on reachingdefinitions analysis.

8 Evaluation

We now present the evaluation of Procurator by answering the following questions. All experiments have been performed on a server with 2 Intel Xeon Silver 4210R CPUs and 128GB of memory.

- Can Procurator effectively detect subtle bugs in existing distributed in-network programs? (§8.1)
- Does Procurator offer advantages over state-of-the-art verifiers for individual P4 programs? (§8.2)
- What about the efficiency of the pruning optimization in Procurator? (§8.3)

We evaluate Procurator using two sets of practical P4 programs. The first set consists of six real-world stateful distributed in-network systems (Table 1), which exhibit common interaction patterns in practice. This set is used to assess Procurator in §8.1 and §8.3. The second set includes four individual P4 programs (Table 2), used for direct comparison with state-of-the-art stateful verifiers for individual programs (§8.2).

8.1 Verification on Distributed Programs

We evaluate Procurator’s effectiveness by detecting bugs in five representative systems (Table 1). Since NetChain’s bug is already discussed in §2.2.2, we focus here on the other four systems. As shown in Table 1, Procurator uncovered 10 bugs in total. All detected bugs were confirmed through error-trace analysis. Notably, most of these bugs are subtle, deeply embedded, and tied to complex interleavings — cases typically beyond the reach of conventional testing or existing P4 verifiers.

P4xos. P4xos implements the Paxos consensus protocol directly in programmable switches, with roles (leader, acceptor, learner) executed in the data plane. The leader manages instance numbers, while acceptors and learners use them as register indices to track consensus rounds. The learner counts

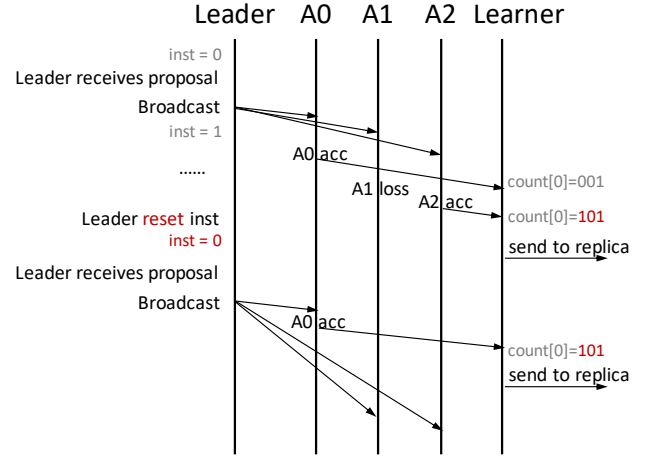


Figure 9: Timing diagram of majority-quorum violation in P4xos. Inst represents the instance number; A0, A1, A2 are the three acceptors; count is a register array where each slot tracks the number of acceptors that have accepted the value for a given instance.

votes and forwards values to replicas once a majority quorum is reached.

Procurator uncovered three bugs in P4xos. Two of them are local implementation errors within individual roles that had already been reported by prior work [28, 40]: one caused by register out-of-bounds access, and another due to the drop_flag not being correctly set. The third bug, however, is a previously unknown violation of the majority-quorum property, which spans multiple distributed nodes. Since this property is fundamental to consensus algorithms, its violation breaks consistency guarantees across nodes. After fixing the two known bugs, Procurator revealed that the system as a whole still violated the majority-quorum property. We now illustrate the root cause of this bug.

Specifically, the learner utilizes registers to maintain vote counts, but lacks a mechanism to clear register values after each instance. Residual vote counts persist across instances, so when the leader resets or wraps the instance number (inst), the learner may reuse stale values. Figure 9 presents a timing diagram illustrating the bug. The system works correctly for

Benchmark	Bug Description	Procurator		p4tv	
		Time (s)	Mem (MB)	Time (s)	Mem (MB)
FRR [8]	Bug 1: Unexpected Mirror	2.4	262	43.57	352
	Bug 2: State Inconsistency	2.1	260	67.14	325
P4NIS [27]	Bug 1: Forwarding Sequence Desync	0.17	178	58.56	309
	Bug 2: Tunnel State Leakage	0.1	140	59.07	306
DDOSD [23]	Vuln: Window Label Collision	0.12	141	109.7	348
Cheetah [5]	Bug: Slot Index Collision	0.22	137	160.6	396

Table 2: Comparative results of bug finding across different P4 verification tools.

inst = 0, but after several rounds, the inst is either wrapped around or explicitly reset by the leader. At this point, the learner inherits the outdated quorum state (101). With only one new vote, it may falsely detect a majority and prematurely forward a value, violating the majority-quorum requirement and leading to consensus incorrectness.

DistCache. We discovered an integer wrap-around bug in DistCache, a multi-switch caching system adapted from Far-Reach [38]. DistCache uses a “power-of-two-choices” scheme to balance load across leaf and spine caches. However, Procurator reveals that the integer counters (leafload, spineload) tracking cache load can overflow under sustained traffic, wrapping to small or negative values.

When this occurs, the proxy switch (*clienttrack*) mistakenly interprets the wrapped value as low load and continues directing traffic to the already overloaded leaf cache, leaving the spine underutilized. The imbalance persists until the counter increments wrap again, prolonging recovery. The bug stems from the lack of explicit overflow handling in load counters, leading to severe traffic skew, low overall throughput, and high query latency.

Gecko. Gecko is a system that leverages programmable P4 switches to reduce the tail latency of distributed key-value stores. Its topology consists of a Master, a Client, a single P4 switch (with two pipelines), and three chunk nodes. A write request proceeds as follows: the Client sends the request to the one pipeline of the switch, which forwards it to the Master. The Master responds to the switch, and the other pipeline then replicates the request to all three chunk servers. Once any two servers acknowledge completion, the switch immediately notifies the Client. A core feature of Gecko is its recirculating *timer* packet, which maintains a global time register to support in-switch timeout detection and “chasing” functionality.

Procurator identified three bugs in Gecko:

Timer Packet Loss or Blockage. If the timer packet is dropped or blocked at the boundary between the switch and the external network, the time register stalls, disabling timeout detection. Pending write requests in registers remain unevaluated, causing inconsistencies and indefinite waiting.

Limited Concurrency Handling. If the Master injects multiple write requests into the first pipeline in quick succession, Gecko’s timer logic updates only one request’s timeout state per pipeline traversal. Under high concurrency, the timer mechanism struggles to keep pace with incoming requests,

leading to inaccurate or delayed timeout detections.

Improper Timer Initialization. If a write request arrives before the timer circuit is active, it bypasses in-switch logic and falls back to Master-handled coordination. This breaks Gecko’s low-latency design and may cause unreliability.

ATP. ATP provides *best-effort* in-network aggregation for distributed training (DT) jobs [22]. It allocates a small portion of programmable switch memory for “aggregators”, indexed by a hash of the (JobID, SequenceNumber) pair. Workers send gradient fragments to ToR switches, which partially aggregate them before forwarding results to the parameter server. ATP includes mechanisms for collision detection (e.g., hash conflicts), fallback to end-host aggregation, and reliability features such as retransmissions on out-of-order acknowledgments. We analyze an open-source P4_16 implementation and Procurator uncovers two issues:

Indefinite Aggregation via 8-bit agtr_time. ATP uses an 8-bit field (p4ml.agtr_time) to track aggregation progress, with a threshold (e.g., 32). If the field exceeds this threshold (e.g., set to 40), the switch continually waits for a completion condition that never occurs. Aggregation is left unfinished, and no response is returned. In our experiments, a host transmitted 40 packets, all of which were silently aggregated but never finalized.

Mismatch under Packet Loss. With packet loss, the switch’s count of completed aggregations sometimes diverges from the actual number of aggregation requests issued. While ATP’s best-effort semantics may tolerate partial aggregation for ML workloads, formally this violates correctness guarantees. In controlled tests with 1000 packets and induced losses, Procurator found states where recorded completions did not match the sender’s intended updates, revealing a latent violation of delivery and aggregation guarantees.

8.2 Verification on Single-Switch Programs

We next evaluate whether Procurator offers advantages over the state-of-the-art stateful P4 verifier, p4tv [52]. We focus on p4tv because it focusing on the task of verifying Linear Temporal Logic properties, and generates execution traces upon verification failures, allowing us to confirm that the reported errors match the bugs we identified. Other verifiers are either stateless [11, 28] or lack trace support and LTL capability [51], which hinders root cause analysis. Since p4tv

System	without slicing			with slicing		
	States	Memory	Time	States	Memory	Time
Netchain	229374	17.45GB	1min39s	229374	7.05GB	50s
P4xos	10765	17.81GB	1min27s	10765	13.86GB	1min7s
Gecko	1348	9.8GB	45s	1348	1.26GB	4.7s
ATP	72	131MB	0.17s	72	40MB	0.07s
DistCach	163858	2.98GB	18.5s	163858	2.91GB	18s

Table 3: Results showing the impact of slicing on memory and execution time.

only supports single-device in-network programs, we conduct experiments on representative ones, including FRR [8], P4NIS [27], Cheetah [5], and DDOSD [23].

Table 2 reports the detected bugs, verification time, and memory consumption of Procurator and p4tv. Both tools detect the same set of bugs across all in-network systems, but Procurator achieves significantly better performance. Specifically, compared to p4tv, Procurator efficiently reduces the memory consumption by $1.9\times$ on Cheetah, and reduces the verification time by $913.2\times$ on DDOSD. By leveraging the Semantic-Aware State Pruner, Procurator generates only the packet sequences necessary to expose a bug, effectively pruning the state space. In contrast, p4tv models incoming packets as non-deterministic within an infinite loop, forcing its SMT-based model checker to exhaustively explore a vast space of packet inputs and interleavings. This exhaustive exploration is computationally expensive, particularly for bugs that require long and specific state-transition sequences.

8.3 Overhead

We now examine Procurator’s overhead by measuring memory usage and execution time while verifying six real-world systems. Table 3 compares results with and without the pruning optimization. The results show that pruning significantly reduces both memory consumption and verification time, highlighting its effectiveness in eliminating redundant or unnecessary components from the system’s state representation. We next analyze Procurator’s verification overhead on individual systems.

The pruning optimization achieves the largest improvements for Gecko, reducing memory usage by 87% and execution time by 90% by removing components irrelevant to the verified properties. This is primarily because Gecko contains many components that are not critical to the specific properties being verified, making pruning highly effective. In contrast, DistCache sees minimal memory reduction, as most of its states are essential. Netchain and P4xos experience moderate gains, with memory reductions of 60% and 22% and execution time reductions of 50% and 23%, respectively. These results indicate that pruning can substantially enhance memory and time efficiency for systems containing a mix of relevant and irrelevant components, though the benefit depends on the proportion of non-essential data.

9 Related Work

We discuss related work to Procurator in this section.

P4 program verification. P4-NoD [31], ASSERT-P4 [12, 33], and Vera [40] translate P4 into other languages (e.g., SEFL) and use *symbolic execution* to explore execution paths and check assertions. p4v [28], bf4 [11], and Aquila [42] improve scalability via *deductive verification*, formally reasoning about weakest pre-conditions to prove correctness. P4Inv [51] partially addresses this by modeling programs as infinite loops and inferring packet invariants. In contrast, Procurator models P4 programs using the actor paradigm and CSP models, capturing infinite-loop behavior, supporting features like recirculation and mirroring, and enabling verification of complex interactions.

Stateful network verification. Stateful network verification is crucial for ensuring the correctness and reliability of modern networks with stateful components like middleware. SymNet [41] uses symbolic execution with packet-header state and introduces SEFL for efficiency. VMN [36] employs SMT solvers and abstracts middleware to reduce network scale and speed up verification. Velner et al. [43] made region traffic isolation verification decidable via packet-order abstraction, later optimized by Alpernas et al. [4]. NetSMC [48] simplifies the network to a single packet model and improves scalability with a custom symbolic model checking algorithm. In contrast, Procurator targets on capturing the internal interactive behaviors in real-world distributed in-network programs.

10 Conclusion

This paper presents Procurator, an efficient verification framework for distributed in-network programs. By modeling pipelines as reactive actors, unifying their interactions as message passing within CSP, and supporting flexible property specification through a unified intent language, Procurator effectively captures interactive behaviors and detects subtle related bugs. To ensure scalability, it incorporates a semantic-aware state pruner that reduces verification complexity. Evaluation results show that Procurator not only uncovers previously overlooked bugs in real-world systems but also achieves a significant reduction in verification time and memory consumption.

References

- [1] Intel Tofino, 2016.
- [2] CVE-2024-0106: Nvidia connectx host firmware for bluefield dpu. <https://nvd.nist.gov/vuln/detail/CVE-2024-0106>, 2024. NVD entry: denial of service, data tampering, and limited information disclosure.
- [3] Gul Agha. *Actors: a model of concurrent computation in distributed systems*. MIT press, 1986.
- [4] Kaley Alpernas, Roman Manevich, Aurojit Panda, Mooly Sagiv, Scott Shenker, Sharon Shoham, and Yaron Velner. Abstract interpretation of stateful networks. In *Static Analysis: 25th International Symposium, SAS 2018, Freiburg, Germany, August 29–31, 2018, Proceedings* 25, pages 86–106. Springer, 2018.
- [5] Tom Barbette, Erfan Wu, Dejan Kostić, Gerald Q Maguire, Panagiotis Papadimitratos, and Marco Chiesa. Cheetah: A high-speed programmable load-balancer framework with guaranteed per-connection-consistency. *IEEE/ACM Transactions on Networking*, 30(1):354–367, 2021.
- [6] Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, Jennifer Rexford, Cole Schlesinger, Dan Talayco, Amin Vahdat, George Varghese, et al. P4: Programming protocol-independent packet processors. *ACM SIGCOMM Computer Communication Review*, pages 87–95, 2014.
- [7] Tommaso Caiazzzi, Mariano Scazzariello, and Marco Chiesa. Millions of low-latency state insertions on asic switches. *Proc. ACM Netw.*, 1(CoNEXT3), November 2023.
- [8] Marco Chiesa, Roshan Sedar, Gianni Antichi, Michael Borokhovich, Andrzej Kamisiński, Georgios Nikolaidis, and Stefan Schmid. Fast reroute on programmable switches. *IEEE/ACM Transactions on Networking*, 29(2):637–650, 2021.
- [9] Edmund M Clarke. Model checking. In *International conference on foundations of software technology and theoretical computer science*, pages 54–56. Springer, 1997.
- [10] Huynh Tu Dang, Pietro Bressana, Han Wang, Ki Suh Lee, Noa Zilberman, Hakim Weatherspoon, Marco Canini, Fernando Pedone, and Robert Soulé. P4xos: Consensus as a network service. *IEEE/ACM Transactions on Networking*, 28(4):1726–1738, 2020.
- [11] Dragos Dumitrescu, Radu Stoenescu, Lorina Negreanu, and Costin Raiciu. bf4: towards bug-free p4 programs. In *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, pages 571–585, 2020.
- [12] Lucas Freire, Miguel Neves, Lucas Leal, Kirill Levchenko, Alberto Schaeffer-Filho, and Marinho Barcellos. Uncovering bugs in p4 programs with assertion-based verification. In *Proceedings of the Symposium on SDN Research*, pages 1–7, 2018.
- [13] Jiaqi Gao, Ennan Zhai, Hongqiang Harry Liu, Rui Miao, Yu Zhou, Bingchuan Tian, Chen Sun, Dennis Cai, Ming Zhang, and Minlan Yu. Lyra: A cross-platform language and compiler for data plane programming on heterogeneous asics. In *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, pages 435–450, 2020.
- [14] Charles Antony Richard Hoare. Communicating sequential processes. *Communications of the ACM*, 21(8):666–677, 1978.
- [15] G.J. Holzmann. The model checker spin. *IEEE Transactions on Software Engineering*, 23(5):279–295, 1997.
- [16] Matthias Jasny, Lasse Thostrup, Tobias Ziegler, and Carsten Binnig. P4db - the case for in-network oltp. In *Proceedings of the 2022 International Conference on Management of Data*, SIGMOD ’22, New York, NY, USA, 2022. Association for Computing Machinery.
- [17] Jinghui Jiang, Xiwen Fan, Zhenpei Huang, Kairui Zhou, Qiao Xiang, Lu Tang, Qiang Li, and Jiwu Shu. Reducing write tail latency of distributed key-value stores using in-network chasing. In *2024 IEEE/ACM 32nd International Symposium on Quality of Service (IWQoS)*, pages 1–10. IEEE, 2024.
- [18] Xin Jin, Xiaozhou Li, Haoyu Zhang, Nate Foster, Jeongkeun Lee, Robert Soulé, Changhoon Kim, and Ion Stoica. NetChain: Scale-Free Sub-RTT coordination. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, pages 35–49, Renton, WA, April 2018. USENIX Association.
- [19] Xin Jin, Xiaozhou Li, Haoyu Zhang, Robert Soulé, Jeongkeun Lee, Nate Foster, Changhoon Kim, and Ion Stoica. Netcache: Balancing key-value stores with fast in-network caching. *SOSP ’17*, New York, NY, USA, 2017. Association for Computing Machinery.

- [20] Peter Kraft, Fiodar Kazhamiaka, Peter Bailis, and Matei Zaharia. {Data-Parallel} actors: A programming model for scalable query serving systems. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 1059–1074, 2022.
- [21] Leslie Lamport. Specifying systems: the tla+ language and tools for hardware and software engineers. 2002.
- [22] ChonLam Lao, Yanfang Le, Kshiteej Mahajan, Yixi Chen, Wenfei Wu, Aditya Akella, and Michael Swift. {ATP}: In-network aggregation for multi-tenant learning. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, pages 741–761, 2021.
- [23] Ângelo Cardoso Lapolli, Jonatas Adilson Marques, and Luciano Paschoal Gaspary. Offloading real-time ddos attack detection to programmable data planes. In *2019 IFIP/IEEE Symposium on Integrated Network and Service Management (IM)*, pages 19–27. IEEE, 2019.
- [24] Jialin Li, Jacob Nelson, Ellis Michael, Xin Jin, and Dan RK Ports. Pegasus: Tolerating skewed workloads in distributed storage with {In-Network} coherence directories. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 387–406, 2020.
- [25] Junru Li, Youyou Lu, Yiming Zhang, Qing Wang, Zhuo Cheng, Keji Huang, and Jiwu Shu. Switchtx: scalable in-network coordination for distributed transaction processing. *Proceedings of the VLDB Endowment*, 15(11):2881–2894, 2022.
- [26] Yifan Li, Jiaqi Gao, Ennan Zhai, Mengqi Liu, Kun Liu, and Hongqiang Harry Liu. Cetus: Releasing p4 programmers from the chore of trial and error compiling. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 371–385, Renton, WA, April 2022. USENIX Association.
- [27] Gang Liu, Wei Quan, Nan Cheng, Ning Lu, Hongke Zhang, and Xuemin Shen. P4nis: Improving network immunity against eavesdropping with programmable data planes. In *IEEE INFOCOM 2020-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, pages 91–96. IEEE, 2020.
- [28] Jed Liu, William Hallahan, Cole Schlesinger, Milad Sharif, Jeongkeun Lee, Robert Soulé, Han Wang, Călin Cașcaval, Nick McKeown, and Nate Foster. P4v: Practical verification for programmable data planes. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on data communication*, pages 490–503, 2018.
- [29] Ming Liu, Liang Luo, Jacob Nelson, Luis Ceze, Arvind Krishnamurthy, and Kishore Atreya. Incbricks: Toward in-network computation with an in-network cache. AS-PLOS ’17, New York, NY, USA, 2017. Association for Computing Machinery.
- [30] Zaoxing Liu, Zhihao Bai, Zhenming Liu, Xiaozhou Li, Changhoon Kim, Vladimir Braverman, Xin Jin, and Ion Stoica. DistCache: Provable load balancing for Large-Scale storage systems with distributed caching. In *17th USENIX Conference on File and Storage Technologies (FAST 19)*, Boston, MA, February 2019. USENIX Association.
- [31] Nuno Lopes, Nikolaj Bjørner, Nick McKeown, Andrey Rybalchenko, Dan Talayco, and George Varghese. Automatically verifying reachability and well-formedness in p4 networks. *Technical Report, Tech. Rep.*, 2016.
- [32] Matheus E. B. Neves and colleagues. Dynamic property enforcement in programmable data planes (p4box). In *IFIP Networking 2019*, 2019.
- [33] Miguel Neves, Lucas Freire, Alberto Schaeffer-Filho, and Marinho Barcellos. Verification of p4 programs in feasible time using assertions. In *Proceedings of the 14th International Conference on Emerging Networking Experiments and Technologies*, pages 73–85, 2018.
- [34] Andrew Newell, Gabriel Kliot, Ishai Menache, Aditya Gopalan, Soramichi Akiyama, and Mark Silberstein. Optimizing distributed actor systems for dynamic interactive services. In *Proceedings of the Eleventh European Conference on Computer Systems*, pages 1–15, 2016.
- [35] P4 Language Consortium. p4c: The p4 reference compiler. <https://github.com/p4lang/p4c>, 2017.
- [36] Aurojit Panda, Ori Lahav, Katerina Argyraki, Mooly Sagiv, and Scott Shenker. Verifying reachability in networks with mutable datapaths. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 699–718, 2017.
- [37] Fabian Ruffy, Tao Wang, and Anirudh Sivaraman. Gauntlet: Finding bugs in compilers for programmable packet processing. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 683–699, 2020.
- [38] Siyuan Sheng, Huancheng Puyang, Qun Huang, Lu Tang, and Patrick PC Lee. {FarReach}: Write-back caching in programmable switches. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, pages 571–584, 2023.
- [39] Erez Shinan et al. Lark: A modern parsing library for python. *Lark: A modern parsing library for python*.

- [40] Radu Stoenescu, Dragos Dumitrescu, Matei Popovici, Lorina Negreanu, and Costin Raiciu. Debugging p4 programs with vera. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*, pages 518–532, 2018.
- [41] Radu Stoenescu, Matei Popovici, Lorina Negreanu, and Costin Raiciu. Symnet: Scalable symbolic execution for modern networks. In *Proceedings of the 2016 ACM SIGCOMM Conference*, pages 314–327, 2016.
- [42] Bingchuan Tian, Jiaqi Gao, Mengqi Liu, Ennan Zhai, Yanqing Chen, Yu Zhou, Li Dai, Feng Yan, Mengjing Ma, Ming Tang, et al. Aquila: a practically usable verification system for production-scale programmable data planes. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, pages 17–32, 2021.
- [43] Yaron Velner, Kalev Alpernas, Aurojit Panda, Alexander Rabinovich, Mooly Sagiv, Scott Shenker, and Sharon Shoham. Some complexity results for stateful network verification. In *Tools and Algorithms for the Construction and Analysis of Systems: 22nd International Conference, TACAS 2016, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2016, Eindhoven, The Netherlands, April 2-8, 2016, Proceedings 22*, pages 811–830. Springer, 2016.
- [44] Qing Wang, Youyou Lu, Erci Xu, Junru Li, Youmin Chen, and Jiwu Shu. Concordia: Distributed shared memory with in-network cache coherence. In *19th USENIX Conference on File and Storage Technologies, FAST 2021, February 23-25, 2021*, pages 277–292, 2021.
- [45] Wenquan Xu, Zijian Zhang, Yong Feng, Haoyu Song, Zhikang Chen, Wenfei Wu, Guyue Liu, Yinchao Zhang, Shuxin Liu, Zerui Tian, et al. Clickinc: In-network computing as a service in heterogeneous programmable data-center networks. In *Proceedings of the ACM SIGCOMM 2023 Conference*, pages 798–815, 2023.
- [46] Parham Yassini, Khaled Diab, Saeed Mahlouljifar, and Mohamed Hefeeda. Horus: Granular {in-network} task scheduler for cloud datacenters. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 1–22, 2024.
- [47] Zhuolong Yu, Yiwen Zhang, Vladimir Braverman, Mosharaf Chowdhury, and Xin Jin. Netlock: Fast, centralized lock management using programmable switches. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication, SIGCOMM '20*, New York, NY, USA, 2020. ACM.
- [48] Yifei Yuan, Soo-Jin Moon, Sahil Uppal, Limin Jia, and Vyas Sekar. {NetSMC}: A custom symbolic model checker for stateful network verification. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 181–200, 2020.
- [49] Lior Zeno, Ang Chen, and Mark Silberstein. In-network address caching for virtual networks. In *Proceedings of the ACM SIGCOMM 2024 Conference*, ACM SIGCOMM '24, page 735–749, New York, NY, USA, 2024. Association for Computing Machinery.
- [50] Lior Zeno, Dan R. K. Ports, Jacob Nelson, and Mark Silberstein. Swishmem: Distributed shared state abstractions for programmable switches. In *Proceedings of the 19th ACM Workshop on Hot Topics in Networks, HotNets '20*, page 160–167, New York, NY, USA, 2020. Association for Computing Machinery.
- [51] Delong Zhang, Chong Ye, and Fei He. P4inv: Inferring packet invariants for verification of stateful p4 programs. In *IEEE INFOCOM 2024-IEEE Conference on Computer Communications*, pages 2129–2138. IEEE, 2024.
- [52] Delong Zhang, Chong Ye, and Fei He. On temporal verification of stateful p4 programs. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, pages 219–235, Philadelphia, PA, April 2025. USENIX Association.
- [53] Yikai Zhao, Wenrui Liu, Fenghao Dong, Tong Yang, Yuanpeng Li, Kaicheng Yang, Zirui Liu, Zhengyi Jia, and Yongqiang Yang. P4lru: Towards an lru cache entirely in programmable data plane. *ACM SIGCOMM '23*, New York, NY, USA, 2023. Association for Computing Machinery.

Appendices

A Abstracting Parsing and Deparsing

We model parsing and deparsing as atomic operations on the validity of headers, represented by the “.valid” flag associated with each header instance.

A.1 Parser Semantics

For the parser, each successful header extraction is modeled by setting the corresponding “.valid” flag to `true`. We formalize the parser as a function $\text{Parse}(p_{\text{raw}}) = p$, transforming a raw packet into a structured one with header validity flags. A header’s validity is set to `true` if the execution path reaches a parser state with an `extract` statement.

$$\frac{p_{\text{raw}} \rightarrow_{\mathcal{P}} s \quad \wedge \quad \text{extract}(h_i) \in \text{body}(s)}{(\text{Parse}(p_{\text{raw}})).h_i.\text{valid} = \text{true}} \quad (8)$$

Here, $p_{\text{raw}} \rightarrow_{\mathcal{P}} s$ denotes that the processing path for packet p_{raw} in parser $\mathcal{P}$ includes the state s , and $\text{extract}(h_i) \in \text{body}(s)$ denotes that an ‘extract’ statement for header h_i is syntactically present in the definition of state s .

A.2 Deparser Semantics

For the deparser, each `emit()` call is modeled as adding the corresponding header to a temporary emission set. Before serialization, a final cleanup step resets the “.valid” flag of any header not in this set, ensuring only explicitly emitted headers remain valid.

We model the deparser’s behavior as a two-phase function, $\text{Deparse}(p) = p_{\text{final}}$.

Phase 1: Deparser Control Block Execution. This phase models the effect of `emit()` calls. We introduce a temporary set, `EmitSet`, which is constructed based on the deparser’s execution trace.

$$\frac{\text{emit}(p.h_i) \in \text{trace}(\text{exec}(\mathcal{D}, p))}{h_i \in \text{EmitSet}} \quad (9)$$

In this rule, $\text{trace}(\text{exec}(\mathcal{D}, p))$ represents the sequence of statements that are actually executed when the deparser control block $\mathcal{D}$ is run on packet p . The premise holds if an `emit` call for header h_i is part of this trace, correctly handling conditional logic. This translates the imperative action of an `emit()` call into membership in the `EmitSet`.

Phase 2: Architectural Cleanup. After all logic in the deparser control block has been executed, a final cleanup step determines the final validity state of each header based on the contents of `EmitSet`. Let $\mathcal{H}_p$ be the set of all header instances defined for packet p .

$$\frac{h_j \in \mathcal{H}_p \wedge h_j \notin \text{EmitSet}}{p_{\text{final}}.h_j.\text{valid} = \text{false}} \quad (10)$$

This rule precisely models the P4 architectural specification: only headers that are explicitly emitted are preserved in the output packet. Any header that was valid before the deparser but was not emitted is implicitly dropped (i.e., its ‘.valid’ bit is cleared). This mechanism is crucial for verifying programs that rely on dynamically modifying packet headers.

B Additional TM Semantic

$$\frac{p.\text{meta.clone_spec} = \text{I2E} \quad p_{\text{clone}} = \text{clone}(p)}{\text{TM}(p) = \text{enq}(\text{enq}([], p), p_{\text{clone}})} \quad (11)$$

This rule models the ingress-to-egress mirror primitive. The TM duplicates the packet and enqueues both the original and the clone into the egress channel. The final disposition of both packets will be determined during the egress-draining phase of the current actor reaction.

$$\frac{p.\text{meta.clone_spec} = \text{I2I} \quad p'_{\text{clone}} = \text{clone_and_mark_i2i}(p)}{\text{TM}(p) = \text{enq}(\text{enq}([], p), p'_{\text{clone}})} \quad (12)$$

This rule models the first step of an ingress-to-ingress clone. The TM duplicates the packet but applies a special mark to the clone. This mark serves as an internal message for the `FinalDispatch` stage. The marked packet transits the egress pipeline merely as a token to unify the model, instructing FD to generate a ‘recirc’ event on its behalf.

$$\frac{p.\text{meta.drop_flag} = \text{true}}{\text{TM}(p) = []} \quad (13)$$

This rule models packet drops within the ingress pipeline. If a packet is marked to be dropped, the TM produces an empty egress queue, effectively terminating the packet’s processing journey before it can enter the egress pipeline.

C Additional FD Semantic

$$\frac{p.\text{meta.is_i2i_clone} = \text{true}}{\text{FD}(p) = ([(I_{\text{self}}, p'), []])} \quad (14)$$

This rule completes the I2I mirror operation by interpreting the internal message from **[TM-I2I-Clone]**. While architecturally the packet bypasses egress, our model centralizes all disposition logic here. This rule sees the marked packet and converts it into a `recirc` event, correctly modeling the end-to-end behavior of a packet re-entering the ingress pipeline.

$$\frac{p.\text{meta.drop_flag} = \text{true}}{\text{FD}(p) = ([], [])} \quad (15)$$

This rule models packet drops within the egress pipeline. If a packet is marked to be dropped at this stage, FD produces an empty set of events. The packet’s processing is silently terminated with no observable external effect.

D Executable Translation for Model Checking

Modeling assumptions and non-blocking guarantees. Our reaction semantics requires that a single pipeline pass (*Ingress-triggered*, *Egress-draining*) executes atomically without internal interleavings. To enforce this in Promela, we adopt the following modeling constraints for the executable model:

(1) *FIFO and bounded channels.* Each device’s ingress channel and each device-local egress queue are FIFO and bounded. We choose per-device bounds Q_IN and Q_EG large enough so that all `!` operations inside a single `atomic{}` do not block (i.e., the egress queue does not overflow; `FinalDispatch` never blocks on an enqueue to an ingress). Violations trigger explicit asserts.

(2) *Local egress queue implements Drain.* Egress draining (Rule (2)(3)) is realized by a device-local queue `egress_q` that is fully emptied before the reaction ends.

(3) *FD is the sole observable exit.* All observable events (our external observations) are produced only by `FinalDispatch` as `!` into some ingress channel, matching the observable alphabet in §4.

(4) *Links resolve forwarding deterministically.* The mapping from `(self, egress_spec)` to the destination’s ingress channel is fixed by the network topology `Links`; if no mapping exists, the packet is dropped at FD (producing no observable event).

Global bounds and packet skeleton. We expose bounds as preprocessor constants and use a minimal packet skeleton that carries only fields read by the translation (`clone/recirc/drop` flags and egress port). These names correspond to the meta fields referenced by Π_I/Π_E and FD in the core semantics.

```
#define Q_IN 8
#define Q_EG 8

mtype = { NO_CLONE, I2E, I2I };
typedef packet_t {
    mtype clone_spec;
    bool drop_flag;
    bool recirc_flag;
    bool is_i2i_clone;
    byte egress_spec;
}
```

Links resolving function. We encode `Links` as a guarded `send_to` helper. This is the only place where a device may send to another device’s ingress.

```
inline send_to(byte self, byte egress_spec
, packet_t p)
{
    if
```

```
:: self == 1 && egress_spec == 10 ->
s2_ingress ! p
:: self == 2 && egress_spec == 20 ->
s1_ingress ! p
:: else -> skip
fi
}
```

Pipeline stubs. The following stubs stand for the (inlined) realizations of Π_I/Π_E over `parser/control/deparser`. The translation emits concrete bodies from the P4 IR; here we present their signatures.

```
inline IngressParser(packet_t p) {}
inline Ingress(packet_t p) {}
inline IngressDeparser(packet_t p) {}
inline EgressParser(packet_t p) {}
inline Egress(packet_t p) {}
inline EgressDeparser(packet_t p) {}
inline get_mirror_port(packet_t p) {}
```

Faithfulness to the abstract semantics. By construction: (i) a single reaction is executed as one `atomic{}` block (*Ingress-triggered*, *Egress-draining*); (ii) Drain is realized by the local `egress_q` loop with the concatenation discipline $L = L_p \cdot L_0$; (iii) all observable effects are `!` into ingress channels at *FinalDispatch*; and (iv) forwarding targets are determined solely by `Links`. Together with the non-blocking bounds Q_IN/Q_EG and the *asserts* above, this executable model is a faithful instance of the formal semantics used in our soundness argument.